GovBiz

AI 기반 정부지원사업 탐색·신청 관리 플랫폼

여러 기관에 분산된 공고를 모아 기업에 맞는 사업을 찾고,
신청 준비부터 진행 관리까지 하나의 서비스에서 연결합니다.

PRODUCT

웹·모바일

QUALITY

평가 / 예산 / 실행 추적

OPERATION

K8s / GitOps / 자동 확장

어떤 문제를 해결하는가

공고를 찾는 과정과 신청을 준비하는 과정을 하나의 흐름으로 연결합니다.

01 탐색의 어려움

기관마다 흩어진 공고
서로 다른 검색·필터 방식
우리 기업과의 적합성 판단

02 근거 확인의 부담

지원 대상·신청 조건 확인
긴 원문과 첨부 서식 검토
추천 이유의 출처 확인

03 준비 과정의 단절

답변·신청 서류 준비
마감·진행 단계 관리
협업 파트너 탐색

공고 탐색 → 공식 근거 확인 → 신청 준비 → 진행·협업 관리

3차 기반과 4차 확장 범위

유지한 기술과 바뀐 구성을 같은 역할끼리 비교합니다.

영역	3차 프로젝트 기반	4차 프로젝트 확장
서비스	공고 검색·근거 답변·신청 준비 중복 검토·협업 파트너 탐색	**Supervisor로 모든 에이전트 기능 통합**
클라이언트	웹 중심 사용자 경험	**Expo 모바일 + 공통 계약·로직 공유**
품질 운영	개별 AI 기능·평가 기록	**평가 접수·예산·검토·실행 추적**
실행 환경	EC2 + Docker Compose	**Kubernetes(EKS) + EC2 노드**
이미지 저장	Amazon ECR	**Amazon ECR (동일)**
배포 반영	SSM으로 EC2의 Compose 갱신	**Argo CD로 Kubernetes 배포 동기화**
자동 확장	자동 확장 구성 기록 없음	**HPA(Pod) + Cluster Autoscaler(EC2)**

서비스 책임을 네 영역으로 분리

사용자 업무, 공식 데이터, AI 추론, 운영 평가가 각자의 계약을 소유합니다.

Core Service

사용자 업무의 진입점

인증·권한, 관심 공고, 신청·협업
AI·Catalog 호출과 결과 검증

Kotlin · Spring Boot · MyBatis

Catalog Service

공식 공고의 수집과 발행

제공처별 수집·정규화
검색 색인 준비 후 snapshot 발행

Kotlin · Spring Boot

AI Service

검색·생성·분류 실행

OpenAI 기반 추론
근거 답변·Supervisor·문서 지원

Python · FastAPI · OpenAI

Ops Service

평가와 운영 정책

평가 명세·예산·검토 상태
Prefect 실행·결과·추적 연결

Python · Django · MySQL

웹과 모바일이 같은 업무 계약을 사용

화면은 각 플랫폼에 맞추고, 공통 DTO·API 계약과 업무 로직은 Shared에 모읍니다.

Web

React 기반 서비스 화면
관리자 LLMOps 화면
Vercel을 통한 웹 제공

Shared

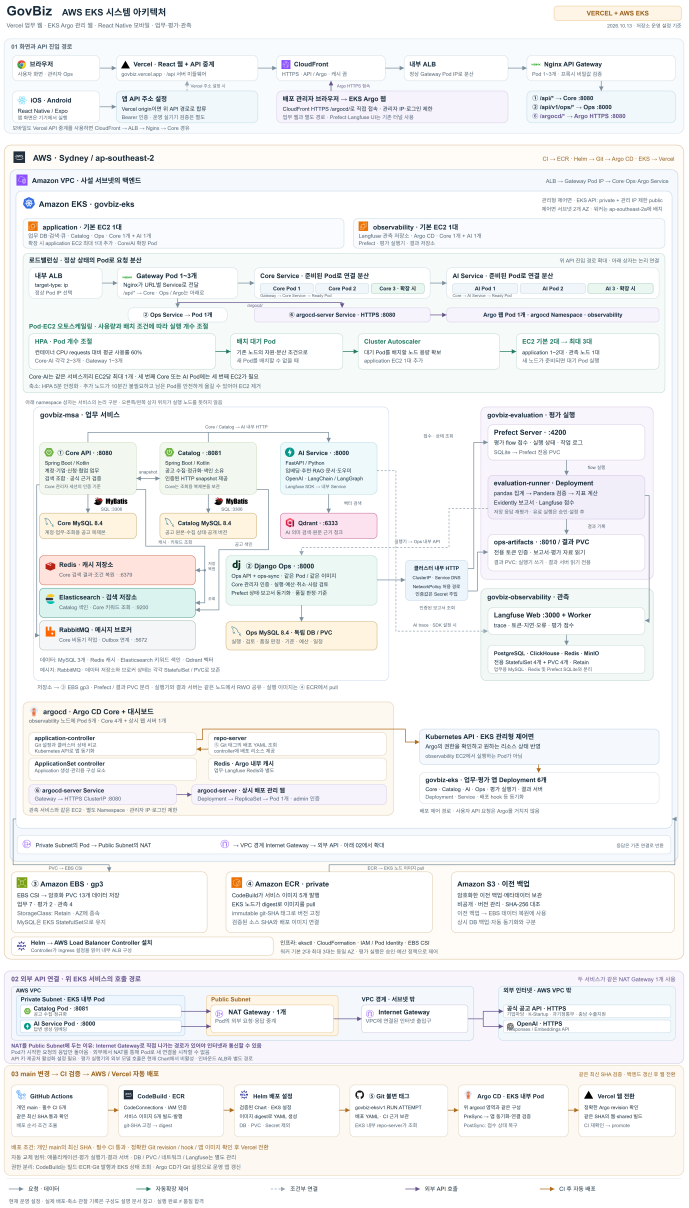
공통 DTO·API 계약
검색·필터 등 공통 업무 로직
웹·모바일에서 함께 사용

Mobile

React Native · Expo
관리자 Gov 에이전트 포함
Firebase App Distribution
팀 테스트 빌드 배포

브라우저와 앱의 사용 경로를 넓히면서, 계약 변경에 따른 차이를 함께 관리합니다.

Firebase App Distribution은 모바일 테스트 배포에 사용합니다.



Vercel · AWS EKS

운영 전체 구성도

웹·모바일의 API 진입 경로

업무·평가·관측 서비스와 데이터 저장소

로드밸런싱·자동 확장·GitOps 배포

운영 구성도 ① 진입·분산·자동 확장

GovBiz AWS EKS 시스템 아키텍처

Vercel 업무 웹 · EKS Argo 관리 웹 · React Native 모바일 · 업무·평가·관측

VERCEL + AWS EKS

2026.10.13 · 저장소 운영 설정 기준

01 화면과 API 진입 경로

- **브라우저** — 사용자 화면 · 관리자 Ops
- **Vercel · React 웹 + API 중계** — govbiz.vercel.app · /api 서버 미들웨어
- **CloudFront** — HTTPS · API / Argo · 캐시 끔
- **내부 ALB** — 정상 Gateway Pod IP로 분산
- **Nginx API Gateway** — Pod 1~3개 · 프록시 비밀값 검증
- **iOS · Android** — React Native / Expo · 앱 화면은 기기에서 실행
- **앱 API 주소 설정** — Vercel origin이면 위 API 경로로 합류 · Bearer 인증 · 운영 실기기 검증은 별도
- **배포 관리자 브라우저 → EKS Argo 웹** — CloudFront HTTPS /argocd/로 직접 접속 · 관리자 IP·로그인 제한 · 업무 웹과 별도 경로 · Prefect·Langfuse UI는 기존 터널 사용
- ① /api/* → Core :8080
- ② /api/v1/ops/* → Ops :8000
- ⑥ /argocd/* → Argo HTTPS :8080

Vercel 주소 설정 시 · Argo HTTPS 접속

모바일도 Vercel API 중계를 사용하면 CloudFront → ALB → Nginx → Core 경유

AWS · Sydney / ap-southeast-2

CI → ECR · Helm → Git → Argo CD · EKS → Vercel

Amazon VPC · 사설 서브넷의 백엔드

ALB → Gateway Pod IP → Core·Ops·Argo Service

Amazon EKS · govbiz-eks

관리형 제어면 · EKS API: private + 관리 IP 제한 public
제어면 서브넷 2개 AZ · 워커는 ap-southeast-2a에 배치

- **application · 기본 EC2 1대** — 업무 DB·검색·큐 · Catalog · Ops · Core 1개 + AI 1개 · 확장 시 application EC2 최대 1대 추가 · Core/AI 확장 Pod
- **observability · 기본 EC2 1대** — Langfuse·관측 저장소 · Argo CD · Core 1개 + AI 1개 · Prefect · 평가 실행기 · 결과 저장소

로드밸런싱 · 정상 상태의 Pod로 요청 분산

위 API 진입 경로 확대 · 아래 상자는 논리 연결

- **내부 ALB** — target-type: ip · 정상 Pod IP 선택
- **Gateway Pod 1~3개** — Nginx가 URL별 Service로 전달 · /api/* → Core · Ops / Argo는 아래로
- **Core Service · 준비된 Pod로 연결 분산** — Core Pod 1 · Core Pod 2 · Core 3 · 확장 시 (Gateway → Core Service → Ready Pod)
- **AI Service · 준비된 Pod로 연결 분산** — AI Pod 1 · AI Pod 2 · AI 3 · 확장 시 (Core → AI Service → Ready Pod)
- ② Ops Service → Pod 1개
- /argocd/
- ⑥ argocd-server Service · HTTPS :8080
- Argo 웹 Pod 1개 · argocd Namespace · observability

Pod·EC2 오토스케일링 · 사용량과 배치 조건에 따라 실행 개수 조절

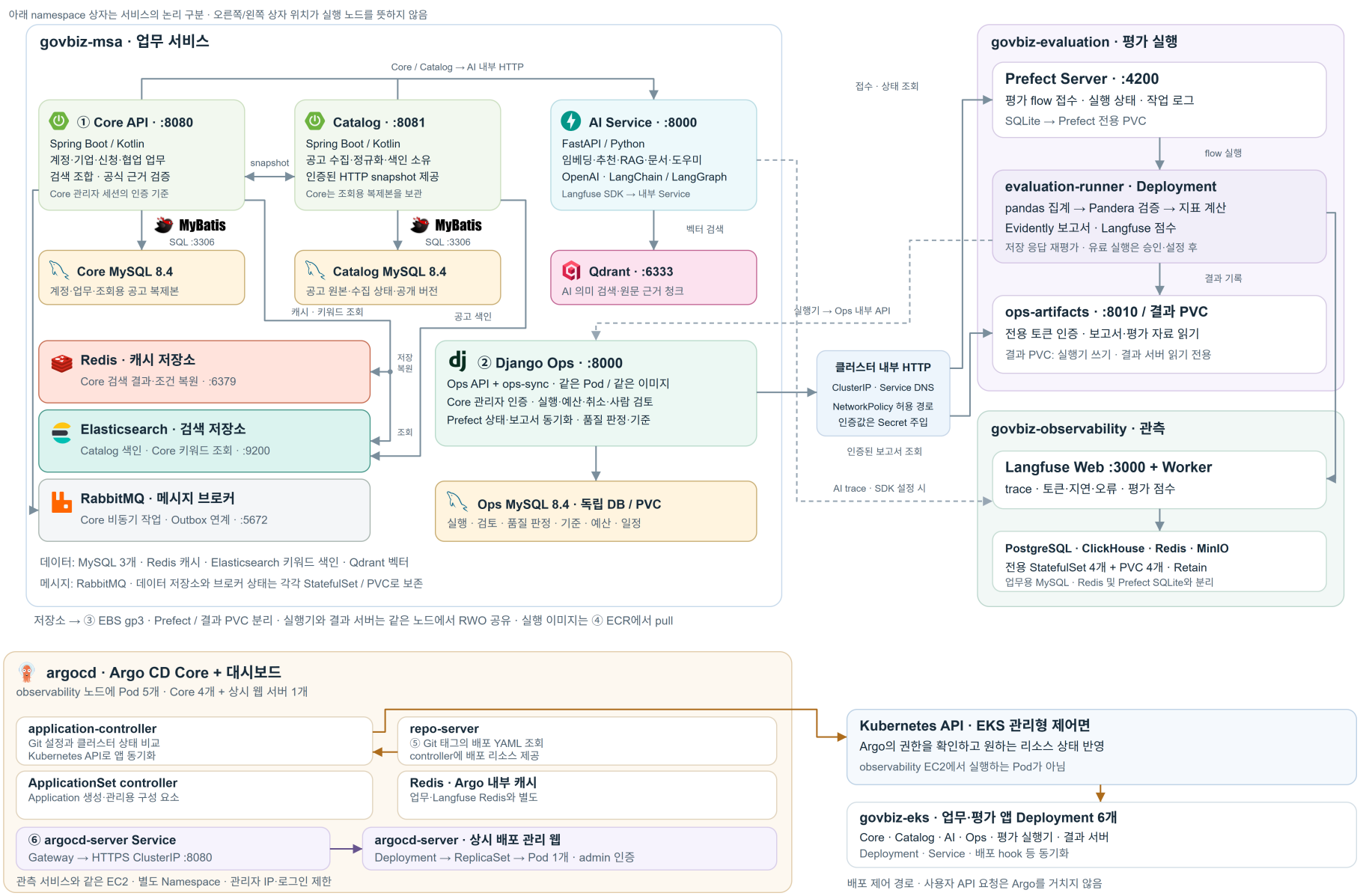
- **HPA · Pod 개수 조절** — 컨테이너 CPU requests 대비 평균 사용률 60% · Core·AI 각각 2~3개 · Gateway 1~3개
- **배치 대기 Pod** — 기존 노드의 자원·분산 조건으로 새 Pod를 배치할 수 없을 때
- **Cluster Autoscaler** — 대기 Pod를 배치할 노드 용량 확보 · application EC2 1대 추가
- **EC2 기본 2대 → 최대 3대** — application 1~2대 · 관측 노드 1대 · 새 노드가 준비되면 대기 Pod 실행

Core·AI는 같은 서비스끼리 EC2당 최대 1개 · 세 번째 Core 또는 AI Pod에는 세 번째 EC2가 필요

축소: HPA 5분 안정화 · 추가 노드가 10분간 불필요하고 남은 Pod를 안전하게 옮길 수 있어야 EC2 제거

아래 namespace 상자는 서비스의 논리 구분 · 오른쪽/왼쪽 상자 위치가 실행 노드를 뜻하지 않음

운영 구성도 ② 업무·평가·관측·Argo CD



운영 구성도 ③ 저장소·외부 API·자동 배포

Private Subnet의 Pod → Public Subnet의 NAT → VPC 경계 Internet Gateway → 외부 API · 아래 02에서 확대

응답은 기존 연결로 반환

PVC → EBS CSI

ECR → EKS 노드 이미지 pull

③ Amazon EBS · gp3

EBS CSI → 암호화 PVC 13개 데이터 저장
업무 7 · 평가 2 · 관측 4
StorageClass: Retain · AZ에 종속
MySQL은 EKS StatefulSet으로 유지

④ Amazon ECR · private

CodeBuild가 서비스 이미지 5개 발행
EKS 노드가 digest로 이미지를 pull
immutable git-SHA 태그로 버전 고정
검증된 소스 SHA와 배포 이미지 연결

Amazon S3 · 이전 백업

암호화한 이전 백업·메타데이터 보관
비공개 · 버전 관리 · SHA-256 대조
이전 백업 → EBS 데이터 복원에 사용
상시 DB 백업·자동 동기화와 구분

Helm → AWS Load Balancer Controller 설치
Controller가 Ingress 설정을 읽어 내부 ALB 구성

인프라: eksctl · CloudFormation · IAM / Pod Identity · EBS CSI
워커 기본 2대·최대 3대는 동일 AZ · 평가 실행은 승인·예산 정책으로 제어

02 외부 API 연결 · 위 EKS 서비스의 호출 경로

두 서비스가 같은 NAT Gateway 1개 사용

AWS VPC

Private Subnet · EKS 내부 Pod
- Catalog Pod · :8081 — 공고 수집·정규화
- AI Service Pod · :8000 — 답변 생성·임베딩

Public Subnet
- NAT Gateway · 1개 — Pod의 외부 요청·응답 중계

VPC 경계 · 서브넷 밖
- Internet Gateway — VPC에 연결된 인터넷 출입구

외부 인터넷 · AWS VPC 밖
- 공식 공고 API · HTTPS — 기업마당 · K-Startup · 과기정통부 · 충남 수출지원
- OpenAI · HTTPS — Responses / Embeddings API

NAT를 Public Subnet에 두는 이유: Internet Gateway로 직접 나가는 경로가 있어야 인터넷과 통신할 수 있음
Pod가 시작한 요청의 응답만 돌아옴 · 외부에서 NAT를 통해 Pod로 새 연결을 시작할 수 없음
API 키·제공처 활성화 설정 필요 · 평가 실행기의 외부 모델 호출은 현재 Chart에서 비활성 · 인바운드 ALB와 별도 경로

03 main 변경 → CI 검증 → AWS / Vercel 자동 배포

같은 최신 SHA 검증 · 백엔드 갱신 후 웹 전환

GitHub Actions
개인 main · 필수 CI 5개
같은 최신 SHA 통과 확인
배포 순서·조건 조율

CodeBuild · ECR
CodeConnections · IAM 인증
서비스 이미지 5개 빌드·발행
git-SHA 고정 → digest

Helm 배포 설정
검증된 Chart · EKS 설정
이미지 digest로 YAML 생성
DB · PVC · Secret 제외

⑤ Git 불변 태그
govbiz-eks/v1.RUN.ATTEMPT
배포 YAML · CI 근거 보관
EKS 내부 repo-server가 조회

Argo CD · EKS 내부 Pod
위 argocd 영역과 같은 구성
PreSync → 앱 동기화·연결 검증
PostSync: 접수 상태 복구

Vercel 웹 전환
정확한 Argo revision 확인
같은 SHA의 웹·shared 빌드
CI 재확인 → promote

배포 조건: 개인 main의 최신 SHA · 필수 CI 통과 · 정확한 Git revision / hook / 앱 이미지 확인 후 Vercel 전환
자동 교체 범위: 애플리케이션·평가 실행기·결과 서버 · DB / PVC / 네트워크 / Langfuse는 별도 관리
권한 분리: CodeBuild는 빌드·ECR·Git 발행과 EKS 상태 조회 · Argo CD가 Git 설정으로 운영 앱 갱신

요청 · 데이터 / 자동확장 제어 / 조건부 연결 / 외부 API 호출 / CI 후 자동 배포

현재 운영 설정 · 실제 배포·축소 관찰 기록은 구성도 설명 문서 참고 · 실행 완료 ≠ 품질 합격

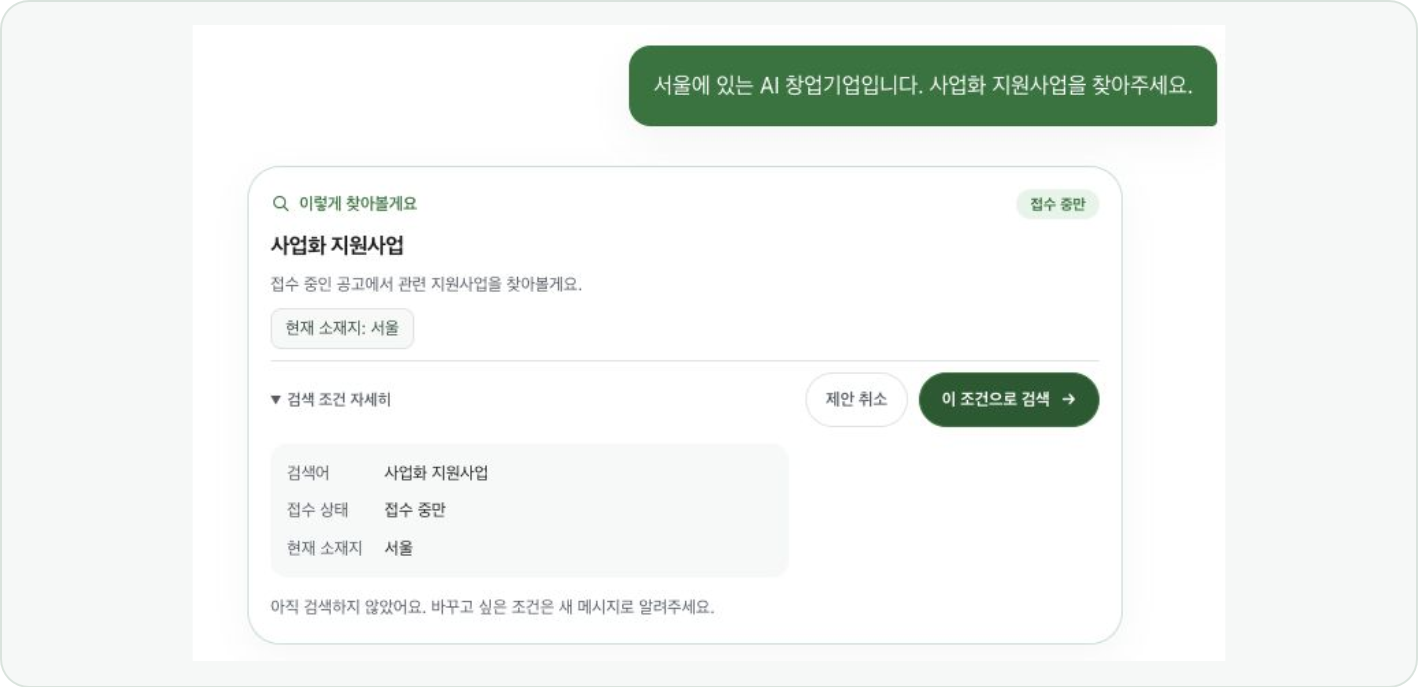
자연어를 확인 가능한 검색 조건으로

AI가 제안한 조건을 사용자가 확인한 뒤, 키워드와 의미 검색을 결합합니다.

조건 확인이 먼저

질문 → 조건 제안 → 사용자 확인

Elasticsearch: 단어·필터 검색
Qdrant: 의미가 비슷한 공고 검색
Core: RRF로 두 결과 결합
AI: 근거를 붙인 추천 설명



AI 검색 로컬 시연 화면 · 2026-10-10

검색 순위는 관련도를 뜻합니다. 최종 지원 자격은 공식 공고의 조건으로 확인합니다.

공식 원문과 답변의 근거를 연결

답변 생성 전후에 원문 식별자와 인용 정보를 확인합니다.

01 근거 준비

공식 HTML 원문 확보
Core에서 정리·분할
Qdrant에 근거 색인

현재 공식 원문 경로:
기업마당 · K-Startup

02 답변 생성

질문에 맞는 근거 검색
Core에서 ID·hash 검증
검증한 문맥을 AI에 전달
OpenAI로 답변 생성

03 출처 확인

답변과 인용 근거 검증
원문으로 다시 확인
근거가 부족하면 상태 구분

검색 결과와 생성 답변의
책임을 나누어 추적

공식 원문 → 검증한 근거 → 답변 → 인용 출처를 연결해 사용자가 다시 확인할 수 있도록 합니다.

관리자 Gov 에이전트의 Supervisor

관리자의 요청 의도를 분류하고, 기존 업무 기능 하나로 연결합니다.

ADMIN 전용

웹·모바일
관리자 대화 진입

OpenAI 의도 분류
→ 지원 기능 선택
→ 필요시 사용자 확인
→ 업무 기능 실행

01 지원사업 검색

02 공식 원문 질문

03 신청 준비

04 중복 지원·수혜 검토

05 협업 파트너 탐색

실행 경계: 관리자 권한 확인 · 한 턴에 하나의 기능 위임 · 지원하지 않는 요청은 명시적으로 안내

원본 서식을 유지하는 신청 문서 작성

공고·서식·답변을 확인한 뒤, 원본의 지정 위치에 사용자 답변을 기록합니다.

확인 후 작성

공고 선택 → 서식 확인
답변 입력 → 사용자 검토
원본 복사본에 답변 기록

HWP: Core의 JVM·hwplib
HWPX·PDF: MCP 도구
revision·hash·기록 위치 검증



시연 결과: 9문항 중 가상 회사명 1개 기입 · 전체 작성 아님

원본 서식 유지 · 사용자가 확인한 답변 기입 · 생성 결과 검토와 다운로드까지 지원합니다.

목적에 따라 구분한 세 가지 평가 방식

저장된 답변 평가와 새로운 모델 호출을 분리해 결과와 비용을 해석합니다.

Replay

저장된 질문·답변을 평가
새 모델 답변 생성 없음

과거 결과 재검토
평가 기준 변경 확인

Fixed-context Live

고정된 근거로 새 답변 생성
모델 호출 비용 발생

같은 문맥에서 모델·설정 비교
생성 단계의 차이 확인

RAG Live

고정된 원문에서 검색·생성
임베딩·답변 호출 비용 발생

검색 결과와 답변을 함께 평가
고정 자료의 검색·생성 비교

비교 기준

같은 데이터·질문·평가 기준·생성 조건에서 결과를 비교합니다.
선택 가능한 모델과 실제로 측정한 모델은 구분합니다.

평가 실행과 운영 정책을 분리

화면의 접수부터 결과 검토까지, 각 단계의 상태와 예산을 관리합니다.

1	**관리 화면**	자료·모델·비교 기준 선택
2	**Ops API**	권한·명세·예산 확인과 예약
3	**Prefect 실행기**	평가 실행·결과 저장
4	**결과 검토**	지표·보고서 확인과 사람 검토

실행 이후도 관리

취소·실패·중단 상태 구분
실행 결과와 후처리 상태 분리
복구 가능한 후처리 재개
사용량·예산 이력 확인
Pandera 검증·Evidently 보고서

COMPLETED는 실행 완료 상태입니다. 답변 품질이 좋다는 판정과는 다릅니다.

처리 기록과 실제 모델 호출을 구분

이벤트 개수를 비용으로 해석하지 않고, 모델 호출의 사용량을 따로 확인합니다.

기록 종류	무엇을 기록하는가	사용량 해석
SPAN	검색·변환·검증 등 일반 처리	모델 비용으로 세지 않음
GENERATION	실제 모델 답변 생성 호출	모델·토큰·비용 확인
EMBEDDING	실제 임베딩 호출	입력 사용량·비용 확인

최신 main의 관리자 추적 화면

/ops/traces → Ops API → 관리자 세션 확인 → 내부 Langfuse 조회 · API 키는 서버에서만 사용

평가 숫자와 답변의 의미를 함께 검토

저장된 소규모 RAG 평가 사례에서도 자동 지표가 놓친 내용이 있었습니다.

실행 범위

공식 공고 2건
질문 H01~H06, 총 6개
RAG v2 1회 실행

임베딩 12회 · 답변 6회
입력 13,677 / 출력 555토큰

자동 지표

검색 Recall 1.0
적용 대상 2개 사례

인용 Recall 1.0 / 2개 사례
답변 상태 일치 1.0 / 6개 사례

내용 검토

H01 답변에서
"중소·중견 제조기업"
자격 조건 누락 지적

AI 검토 결과를
사람의 정답 확정과 구분

이 수치는 저장된 6개 질문의 사례입니다. 현재 전체 모델·공고의 정확도나 운영 성능을 대표하지 않습니다.

EKS 운영의 외부 요청 경로

공개 진입점과 클러스터 내부 서비스를 분리합니다.

웹·앱 API 요청

웹·모바일 사용자 요청 → **Vercel** /api 중계 → **CloudFront** HTTPS · VPC origin → **내부 ALB** Gateway로 분산 → **Nginx Gateway** Core·Ops로 프록시

Gateway → Core	사용자 업무 API · Core에서 AI·Catalog와 내부 통신
Gateway → Ops	평가·관리자 추적 API · 운영 정책과 평가 상태 확인

Argo CD는 /argocd/로 관리합니다. GovBiz 내부 추적 화면과 별도 Langfuse 대시보드는 인증 경로가 다릅니다.

두 역할의 노드에 워크로드를 배치

노드 역할과 namespace를 구분하고, Core·AI는 양쪽 노드에 분산합니다.

Application 노드 그룹

업무 DB·검색 저장소
Catalog · Ops
Gateway 배치 가능
Core · AI 기본 실행 Pod

Observability 노드 그룹

Langfuse · Argo CD
Prefect · 평가 실행기 · 결과 저장소
Gateway 배치 가능
Core · AI 기본 실행 Pod

기본 의도: Core와 AI를 노드별 1개씩

namespace는 논리적 관리 경계입니다. 노드 역할과 같은 개념이 아니며, DB를 API Pod마다 복제하지 않습니다.

로드밸런싱과 자동 확장의 세 단계

요청 분산, Pod 개수 조절, EC2 용량 확장이 서로 다른 역할을 맡습니다.

01 요청 분산

ALB → 정상 Gateway
Kubernetes Service
→ 준비된 API Pod

트래픽을 실행 중인
서비스 인스턴스로 분산

02 HPA

CPU requests 대비 평균 60%
Core 2~3개 · AI 2~3개
Gateway 1~3개

Core·AI는 노드 분산을 강제
세 번째 Pod는 새 노드 필요

03 Cluster Autoscaler

Application EC2 1~2대
Observability EC2 1대
전체 설정 범위 2~3대

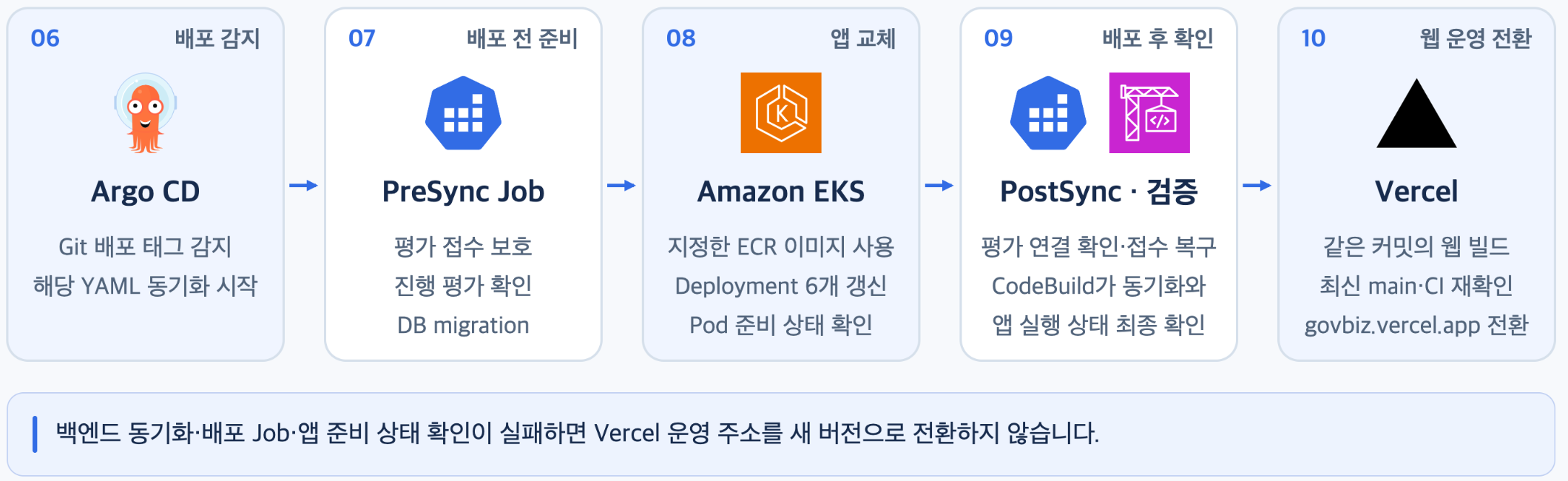
배치가 불가능한 Pod 발생 시
추가 노드를 준비

설정 범위와 현재 대수는 구분합니다. 최근 문서 기록은 3대이며, 2대로 자동 복귀한 최종 확인은 남아 있습니다.

검증한 커밋을 GitOps로 배포

필수 CI와 이미지 발행, Argo CD 동기화가 같은 배포 대상을 가리키도록 연결합니다.

대상 SHA의 필수 CI 성공 → CodeBuild → ECR 이미지 5종
고정 digest의 Helm YAML → 불변 Git 태그 → Argo CD



운영 안전장치와 남은 검증 범위

구현된 보호 장치와 실제 운영에서 더 확인할 항목을 구분합니다.

구현된 안전장치

관리자 권한·평가 예산 확인
실패·취소·복구 상태와 이력 관리
배포 전 접수 중단·평가 종료 확인
DB migration 후 연결 확인·접수 재개
실패한 동기화에서는 전환 차단

남은 운영 과제

단일 AZ·단일 DB의 가용성 한계
실부하 기준 병목·자원 예약량 검토
3대 → 2대 노드 축소 최종 확인
더 넓은 질문·모델의 사람 검토 평가
모바일 실제 기기·운영 경로 확인

Core·AI의 순차 교체와 Catalog·Ops의 단일 실행 보호는 다릅니다. 전체 시스템 무중단을 단정하지 않습니다.

4차 프로젝트가 만든 변화

AI 기능 구현에서 사용자 경험·품질 관리·서비스 운영까지 범위를 확장했습니다.

서비스 연결

공식 공고 → 근거 → 신청 준비
웹·모바일 + 관리자 Supervisor

품질의 가시화

평가 모드·예산·검토 상태
실제 모델 호출과 사용량 추적

운영의 자동화

EKS·Argo CD·ECR 배포
로드밸런싱 + Pod·노드 확장

다음 단계

실사용 평가 표본 확대 · 부하·장애 복구 확인 · 운영 비용과 자원 최적화

Q&A